

Complete EFX₀ Allocations Exist When Every Agent Values at Most Three Goods

Evan Lin

University of Illinois Urbana-Champaign

Abstract. An allocation of indivisible goods is EFX₀ if no agent envies another agent's bundle after the removal of any single good from it, even a good that the envious agent values at zero. We prove that every instance with nonnegative additive valuations in which each agent positively values at most three goods has a complete EFX₀ allocation, for any number of agents. The proof is constructive. After agents that value at most two remaining goods or have a dominant good are peeled off, a serial dictatorship with a careful placement of the unpicked goods, followed if needed by one rotation of picks along a chain of agents, gives an EFX₀ allocation in which at most one bundle has more than two goods. The resulting algorithm uses at most $400(n+m+1)^4$ counted operations. The existence theorem, the structural theorem, the algorithm and its running-time bound are machine-checked in Lean 4, with core Lean and the standard axioms only. Bundles of at most two goods do not always suffice.

Keywords: Fair division · Indivisible goods · EFX · Serial dictatorship · Formal verification · Lean

1 Introduction

Envy-freeness cannot always be achieved with indivisible goods, and envy-freeness up to any good is one of its most studied relaxations [4]. Let agents $N = [n]$ share goods $M = [m]$, with additive valuations $v_i(S) = \sum_{g \in S} v_i(g)$, $v_i(g) \geq 0$. An allocation $X = (X_1, \dots, X_n)$, a partition of M , is *EFX₀* if

$$v_i(X_i) \geq v_i(X_j \setminus \{h\}) \quad \text{for all } i \neq j \text{ and all } h \in X_j,$$

and *EFX* if this is required only for goods h with $v_i(h) > 0$; the stronger notion is usually called EFX₀ [4]. The difference matters when agents value few goods: if X_j contains a good worthless to i , then EFX₀ requires $v_i(X_i) \geq v_i(X_j)$. In particular, goods that nobody values cannot be handed to an arbitrary agent.

Complete EFX allocations are known to exist for additive valuations only in special cases, among them three agents [5], $m \leq n + 3$ [12], four

agents with at most nine goods [3], when every good is valued by at most two agents [6,1,7,13], and on hypergraphs of girth at least four [8]; these are EFX_0 results. For monotone valuations EFX can fail [2,11]. Call an instance *k-relevant* if every agent positively values at most k goods. For $k \leq 2$, serial dictatorship gives an EFX_0 allocation [9]. For $k \leq 4$, Viswanathan and Mehta [14] announce ordinary EFX, with a proof sketch; their algorithm gives the goods nobody values to an arbitrary agent, which can break EFX_0 . We ask whether every 3-relevant instance has a complete EFX_0 allocation. A counterexample would refute the additive EFX conjecture, since perturbing all values by a small $\varepsilon > 0$ (compare [5, Lemma 1]) turns an instance without EFX_0 allocations into a positive one without EFX allocations. But the perturbation destroys the relevance structure, and we know of no result that implies a positive answer (the literature search is recorded in [10]). We prove:

Theorem 1 (existence). *Every instance with $n \geq 1$ agents and non-negative real additive valuations in which every agent positively values at most three goods has a complete EFX_0 allocation.*

Call an agent *balanced* if it values exactly three goods and none of them more than the other two together.

Theorem 2 (one large bundle). *In every instance with $n \geq 1$ agents and nonnegative real additive valuations in which every agent is balanced, some EFX_0 allocation has at most one bundle with more than two goods.*

The construction, LB^+ (Section 3), is a serial dictatorship with a careful placement of the unpicked goods and at most one *rotation* of picks along a chain of agents. It gives a polynomial-time algorithm (Section 4), and everything is machine-checked in Lean 4 (Section 5). A large bundle is sometimes necessary, and may need four goods (Proposition 3).

Status of the claims. The work was done in a public repository [10] whose ledger gives every claim a status: PROVED (complete written proof), CERTIFIED (exhaustive computation over a provably complete finite set, with independently checked certificates), EVIDENCE, CONJECTURE or REFUTED. All positive results below are PROVED. Two independent reviews of the written proof of Theorems 1 and 2 found no error.

2 Preliminaries

Let $R_i = \{g : v_i(g) > 0\}$. Since $\max_{h \in X_j} v_i(X_j \setminus \{h\}) = v_i(X_j) - \min_{h \in X_j} v_i(h)$, a bundle of one good is never strongly envied, and i must

not envy at all a bundle that contains a good outside R_i . If $|R_i| = 3$ we write $R_i = \{a_i, b_i, c_i\}$ with $v_i(a_i) \geq v_i(b_i) \geq v_i(c_i) > 0$, fix this order once (ties broken arbitrarily) as $a_i \succ_i b_i \succ_i c_i$, and call i *balanced* if $v_i(a_i) \leq v_i(b_i) + v_i(c_i)$, *strictly balanced* if the inequality is strict, and *top-heavy* if $v_i(a_i) \geq v_i(b_i) + v_i(c_i)$.

Lemma 1 (peeling). *Let $i \in N$ and $P \subseteq M$ satisfy (i) $v_i(P) \geq v_i(M \setminus P)$ and (ii) $|P| \leq 1$ or P is worthless to every other agent. If the instance without i and P has an EFX₀ allocation Y , then Y together with $X_i = P$ is EFX₀.*

Rule R1 applies Lemma 1 with $P = \{p\}$, p a favourite remaining good of i , when $v_i(p) \geq v_i(M \setminus \{p\})$ (that is, i values at most two remaining goods or is top-heavy), and with $P = \emptyset$ when i values no remaining good. *Rule R2* gives i its goods that no other agent values, if there are at least two and (i) holds. Goods valued by nobody go to an unenvied agent after envy cycles are rotated away (Lemma 4). What remains when none of these applies is a single agent or a *core*: at least two agents, each valuing exactly three goods, strictly balanced, with at most one private good, and no good valued by nobody.

Proposition 1 (core reduction). *If every core with at most N_0 agents has an EFX₀ allocation, so does every 3-relevant instance with between 1 and N_0 agents.*

Cores are the objects of our computations (Section 5). For Theorem 1, rule R1 alone suffices: if R1 fails for every agent's favourite good p , then $2v_i(g) \leq 2v_i(p) < v_i(M)$ for all g , so every agent of a 3-relevant instance values exactly three goods and is strictly balanced, and Theorem 2 allows goods valued by nobody and several private goods.

Lemma 2 (real values reduce to naturals). *If v is nonnegative real additive with $|R_i| \leq 3$ for all i , there are natural-number additive valuations w with the same relevant goods such that $v_i(S) \leq v_i(T) \iff w_i(S) \leq w_i(T)$ for every agent i and all $S, T \subseteq M$.*

Such comparisons are fixed by the ties among a_i, b_i, c_i and the sign of $v_i(a_i) - v_i(b_i) - v_i(c_i)$, and EFX₀, relevance and balance are made of them. Proofs are in Appendix A.

3 The Construction LB⁺

In this section every agent values exactly three goods and is balanced; goods valued by nobody and agents with several private goods are allowed.

Phase 1: serial dictatorship with R1 priority. Agents are processed one at a time; G is the set of goods not yet picked, initially M . The processed agent i takes its $\succ_i$ -favourite good of $R_i \cap G$ as its *pick* Y_i , or no pick ($Y_i = \perp$) if $R_i \cap G = \emptyset$. If some unprocessed agent has $|R_i \cap G| \leq 2$, the next agent is any such agent (an *R1 step*). Otherwise the next agent is any unprocessed agent, and it takes its top (an *insertion step*). The goods never picked form the *junk* J_0 . A *block* is an insertion step with the R1 steps that follow it; its inserted agent is its *leader*. We use four immediate invariants (Appendix B): (I1) if $g \in R_i$ is above Y_i (or $Y_i = \perp$), then g was picked before i 's turn, and (B) by an agent of i 's block; (I2) if $g \in R_i \cap J_0$, then $Y_i \succ_i g$; (I3) an agent that finds all its goods available is a leader.

Pre-allocations. A *pre-allocation* $P = (Y, U)$ has picks $Y_i \in R_i \cup \{\perp\}$ (distinct) and a set U of *upgraded* agents; each $u \in U$ has $Y_u = b_u$ and also holds c_u , and these goods c_u are nobody's pick. The *junk* J is the set of the other goods. For $i \notin U$ the *needs* are $N_i = \{g \in R_i : g \succ_i Y_i\}$ ($N_i = R_i$ if $Y_i = \perp$), and $NA = \bigcup_{i \notin U} N_i$ is the set of goods *needed alone*. P is *valid* if (V1) $J \cap NA = \emptyset$ and (V2) $b_u, c_u \notin NA$ for $u \in U$. An agent $i \notin U$ is *frozen* if $Y_i \in NA$ and a *terminal* otherwise. The *slots* are $\text{cap}(i) = 0$ for upgraded and frozen agents, and 1 or 2 for a terminal with or without a pick; $S = \sum_i \text{cap}(i)$ and $\omega = |J| - S$. The *base* of i is $\{Y_i\}$ (or $\emptyset$) for $i \notin U$ and $\{b_i, c_i\}$ for $i \in U$. A *completion* with owner o (a terminal or upgraded agent) gives every agent its base, at most $\text{cap}(i)$ junk goods to each terminal $i \neq o$, and the remaining junk to o ; without owner, all junk goes into slots. It satisfies the *owner constraint* (OC) if no agent $x \notin U$, $x \neq o$, with $Y_x = a_x$ has $\{b_x, c_x\} \subseteq X_o$.

Theorem 3 (soundness). *Let P be valid. Every completion of P that satisfies (OC) is EFX_0 for every balanced additive valuation consistent with the rankings, and only the owner's bundle can have more than two goods.*

Proof. Only X_o can exceed two goods. Fix i and $j \neq i$ with $|X_j| \geq 2$. Then j is not frozen, so $X_j \cap NA = \emptyset$ by the definition of frozen agents, (V1) and (V2). If $i \in U$, then X_j meets R_i in at most a_i , and $v_i(X_j \setminus \{h\}) \leq v_i(a_i) \leq v_i(b_i) + v_i(c_i) \leq v_i(X_i)$. If $Y_i = \perp$, then $R_i = N_i \subseteq NA$ and X_j is worthless to i . Otherwise every good of $R_i \cap X_j$ is outside $N_i \cup \{Y_i\}$, hence below Y_i . If there is at most one, $v_i(X_j \setminus \{h\}) \leq v_i(Y_i) \leq v_i(X_i)$. If there are two, then $Y_i = a_i$ and $R_i \cap X_j = \{b_i, c_i\}$; for $|X_j| = 2$ removing one leaves a good worth at most $v_i(a_i)$, and $|X_j| \geq 3$ means $j = o$, excluded by (OC). $\square$

The *exposed* agents of an owner o are $E_o = \{x \notin U, x \neq o : Y_x = a_x, \{b_x, c_x\} \subseteq J \cup \text{base}(o)\}$. By filling the other slots with the goods of H first, one gets:

Lemma 3 (owner criterion). *Let P be valid, $\omega \geq 1$. If no $x \in E_o$ has $\{b_x, c_x\} \subseteq \text{base}(o)$, and some $H \subseteq J$ with $|H| \leq S - \text{cap}(o)$ meets $\{b_x, c_x\}$ for every $x \in E_o$, then P has a completion with owner o satisfying (OC).*

Then o is a *valid owner*. If $\omega \leq 0$, the completion without owner has all bundles of at most two goods.

Upgrades and the natural owner. After Phase 1, let $U = \emptyset$; while some $k \notin U$ has $Y_k = b_k, c_k \in J$ and $b_k \notin NA$, add k to U (in any order). The result is valid, by (I2) and because NA only shrinks (Lemma 5). The construction LB of [10] now searches for any valid owner and fails if there is none. LB⁺ tries the last-processed agent $r \notin U$, a terminal. Every frozen agent x has a *need chain* $x = x_0, \dots, x_t$ of distinct agents in which each $x_{s+1} \notin U$ needs Y_{x_s} , $x_0, \dots, x_{t-1}$ are frozen and x_t is a terminal; by (B) it moves forward inside x 's block.

Theorem 4 (the owner r). *Let $\omega \geq 1$ after Phase 1 and the upgrades. Then r is a valid owner unless the bad case holds: an agent k^* of r 's block is exposed for r and frozen, every need chain from k^* ends at r , and the sets $\pi_x = \{b_x, c_x\} \cap J$ ($x \in E_r$) are pairwise disjoint.*

Proof (idea). An exposed agent found all its goods available (b_x, c_x are junk or r 's later pick), so by (I3) it leads its block, and each block has at most one. For each exposed x outside r 's block, x or the end of a need chain from x is a terminal of x 's block. These terminals are distinct and differ from r , so $S - \text{cap}(r) \geq |E_r| - 1$, and $\geq |E_r|$ if no k^* exists. One junk good per set π_x , sharing a good where two sets meet, fits unless the bad case holds (Appendix B). $\square$

Theorem 5 (rotation). *In the bad case, take any need chain $k^* = x_0, \dots, x_t = r$. Let each x_s ($s \geq 1$) take $Y_{x_{s-1}}$, and let k^* take b_{k^*}, c_{k^*} and join U (Y_r is released). The result P' is valid, and if $\omega' \geq 1$, then k^* is a valid owner of P' .*

Proof (idea). Chain agents receive goods they needed, so NA shrinks; $b_{k^*}, c_{k^*} \in J \cup \{Y_r\}$ are free. Every agent exposed for k^* in P' was exposed for r (not the interior of the chain, and not r , by (I1) and the stopping rule of the upgrades), and disjointness keeps its pair out of $\{b_{k^*}, c_{k^*}\}$. The terminals found for the other exposed agents lie outside r 's block and keep their slots, which suffice (Appendix B). $\square$

Construction LB^+ . Run Phase 1 (any choices) and the upgrades (any order). If $\omega \leq 0$, return the completion without owner. Else, if r is a valid owner, return a completion with owner r . Else, by Theorem 4, the bad case holds: rotate along any need chain from k^* to r and return a completion of P' , without owner if $\omega' \leq 0$ and with owner k^* otherwise.

Theorem 6 (LB^+ never fails). *If every agent values exactly three goods and is balanced, then for every choice in Phase 1, in the upgrades and of the chain, LB^+ returns an allocation that is EFX_0 for every balanced additive valuation consistent with the rankings, with at most one bundle of more than two goods.*

Proof. The pre-allocation is valid, and the completion satisfies (OC), by Lemma 3 and Theorems 4 and 5. Apply Theorem 3. $\square$

Proof (of Theorems 1 and 2). Theorem 2 is Theorem 6. For Theorem 1, induct on n : one agent takes everything; if R1 applies to an agent, peel it (Lemma 1); otherwise every agent is balanced (Section 2) and Theorem 2 applies. $\square$

The proof uses no computation. The large bundle has $|NA| - (2n - m) + 2$ goods (a written remark, not formalized).

Example 1. Let agents 0, 1, 2 rank $g_0 \succ g_2 \succ g_3, g_2 \succ g_1 \succ g_4, g_0 \succ g_2 \succ g_1$, processed in the order 1, 0, 2. Agent 1 is inserted and picks g_2 ; agents 0 and 2 then pick g_0 and g_1 in R1 steps. Now $J = \{g_3, g_4\}$, $NA = \{g_0, g_2\}$, agents 0, 1 are frozen, and $r = 2$ has one slot: $\omega = 1$. The bundle $\{g_1, g_3, g_4\}$ for r violates (OC) for agent 1 ($b_1 = g_1, c_1 = g_4$), so LB fails on this run. The rotation along the chain $1 \rightarrow 2$ gives $\{g_0\}, \{g_1, g_4\}, \{g_2, g_3\}$, which is EFX_0 .

4 Algorithm and Running Time

Algorithm $K3ALG$. Stage R: while at least two agents and some goods remain, remove the first agent to which rule R1 applies, with its favourite good or nothing; a lone remaining agent takes all goods left. Stage L: if agents remain, each values exactly three remaining goods and is strictly balanced; sort each agent's goods and run LB^+ with the processing order “the first agent with at most two goods left, else the first agent”, upgrades by index, the owner test below, and a need chain found by scanning forward from k^* (pseudocode in Appendix C).

Theorem 7 (algorithm). *On every instance with $n \geq 1$ agents and natural-number values in which every agent values at most three goods, K3ALG returns an EFX₀ allocation. On every instance with $n \geq 1$ agents it performs at most $400(n + m + 1)^4$ counted operations.*

Correctness follows from Lemma 1 and Theorem 6. The one step that is not obviously polynomial is the owner test: a minimum set of junk goods meeting all sets π_x (each of at most two goods) is a vertex cover. For r it is not needed.

Proposition 2 (the owner test). *With $\omega \geq 1$ after Phase 1 and the upgrades, r is a valid owner if and only if $|E_r| \leq S - \text{cap}(r)$ or two of the sets π_x ($x \in E_r$) meet.*

Proof. The proof of Theorem 4 gives $S - \text{cap}(r) \geq |E_r| - 1$, and nonempty sets π_x have a hitting set of fewer than $|E_r|$ goods if and only if two of them meet. $\square$

In Lean, K3ALG is a program in a *cost monad* (a value paired with an operation count), and `EFX.K3.algo` is defined as its value, so the bound is about the program that computes the allocation. One unit is a list step, a comparison of agents or goods, a read of an input value or of a table, or a comparison or addition of natural numbers of any size. The dominant term is the upgrade loop. Finer bounds, $O(n^4 + n^2m)$ and $O((n+m) \log n)$ after reading the input for a Python implementation, are written but not formalized; that implementation agrees with the Lean program on every instance tested and takes about 1.6 s with 10^5 agents (evidence). K3ALG compares only subset sums of one agent’s values, so by Lemma 2 it runs verbatim on real values (a written remark, not in the ledger).

5 Machine-Checked Proof and Computational Certification

Lean. The formalization uses core Lean 4 (v4.34.0) and no packages. No file contains `sorry`, the build is warning-free, and every declaration depends only on `propext`, `Classical.choice` and `Quot.sound`; a script checks this with a `#print axioms` certificate per ledger-facing theorem, a whole-library axiom check and Lean’s replay checker. The trusted base is six short definitions copied from [9] (Appendix D): values $v: \text{Fin } n \rightarrow \text{Fin } m \rightarrow \mathbb{N}$, an allocation as a map from goods to agents, and EFX₀ with the removed good ranging over *every* good of the other bundle. Theorem 1

is `EFX.target`, Theorem 2 is `EFX.LB.corollaryD` (balance as $2v_i(g) \leq v_i(M)$), and Theorem 7 is `EFX.K3.algo_efx0` with `EFX.K3.algoC_cost`. Via Lemma 2 (`EFX.l12`), `EFX.target_ordered` and `EFX.corollaryD_ordered` extend the first two to values in any linearly ordered cancellative commutative monoid; that $\mathbb{R}_{\geq 0}$ is one is the textbook fact, as core Lean has no reals. Every result of Section 3 is formalized, Theorem 6 over all choices of LB^+ (Appendix D); the final statements use only the trusted base. An audit restated Theorems 1 and 2 independently, before reading the model, and derived them from the main theorems; a second kernel (nanoda) re-checked their natural-number development. A second Lean proof of Theorem 2 for cores, through Pareto-maximal pre-allocations and independent of LB^+ , is also in [10]. Not formalized: the size of the large bundle, the finer running-time bounds and the real-value remark on K3ALG.

Computation. Small cases are also certified independently of the proof (Appendix E). In a core, EFX_0 depends only on the agents’ rankings (Lemma 6), and for every connected core (enumerated by nauty’s `genbg` and cross-checked) stored allocations cover all 6^n ranking profiles, as a SAT-free checker verifies from the raw EFX_0 definition. This certifies EFX_0 for all 3-relevant instances with $n \leq 7$, and Theorem 2 for all cores with $n \leq 6$ and all connected cores with $n = 7$, or $n = 8$ and $m \geq 13$. Computation also refuted stronger shapes; Proposition 3 gives hand-checked counterexamples.

6 Conclusion and Open Problems

Every 3-relevant instance has a complete EFX_0 allocation, found in polynomial time, with a machine-checked proof. The case of four relevant goods per agent is open; the repository has work in progress on it, about which we claim nothing. Also open: whether LB, without rotation, never fails; the minimum size and canonical content of the large bundle (conjecture K of [10]); and the complexity of the owner test for arbitrary owners.

Acknowledgments. The proofs, code and Lean formalization were developed with the assistance of AI coding agents (Claude Code). Every claim in this paper is machine-checked, computer-certified or refereed, as stated at the claim.

References

1. Afshinmehr, M., Ashuri, A., Mahmoudkhan, P., Mehlhorn, K., Shahrezaei, A.M.: EFX allocations exist on multi-graphs. arXiv:2606.18665 (2026)
2. Akrami, H., Mayorov, A., Mehlhorn, K., Srinivas, S., Weidenbach, C.: A counterexample to EFX $n \geq 3$ agents, $m \geq n + 5$ items, submodular valuations via SAT-solving. arXiv:2604.18216 (2026)
3. Alkassar, E., Fouz, M., Mehlhorn, K.: Complete EFX allocations exist for four additive agents and up to nine goods. arXiv:2608.08590 (2026)
4. Amanatidis, G., Birmpas, G., Filos-Ratsikas, A., Voudouris, A.A.: Fair division of indivisible goods: A survey. arXiv:2202.07551 (2022)
5. Chaudhury, B.R., Garg, J., Mehlhorn, K.: EFX exists for three agents. arXiv:2002.05119 (2020)
6. Christodoulou, G., Fiat, A., Koutsoupias, E., Sgouritsa, A.: Fair allocation in graphs. In: Proceedings of the 24th ACM Conference on Economics and Computation (EC 2023). pp. 473–488 (2023). <https://doi.org/10.1145/3580507.3597764>
7. Kaviani, A., Seddighin, M., Shahrezaei, A.: Almost envy-free allocation of indivisible goods: A tale of two valuations. arXiv:2407.05139 (2024)
8. Lianas, T., Sgouritsa, A., Sotiriou, M.M.: EFX allocation in (multi)hypergraphs. arXiv:2608.03171 (2026)
9. Lin, E.: mrd-efx. GitHub repository (Lean formalization for at most two relevant goods per agent), <https://github.com/evanlin23/mrd-efx>
10. Lin, E.: efx0-three-goods: EFX₀ with at most three relevant goods per agent (proofs, certificates, Lean formalization). GitHub repository, <https://github.com/evanlin23/efx0-three-goods> (2026)
11. Mackenzie, S., Suzuki, M.: Counterexamples to EFX for submodular and subadditive valuations. arXiv:2605.06451 (2026)
12. Mahara, R.: Extension of additive valuations to general valuations on the existence of EFX. arXiv:2107.09901 (2021)
13. Sgouritsa, A., Sotiriou, M.M.: On the existence of EFX allocations in multigraphs. arXiv:2502.09777 (2025)
14. Viswanathan, M., Mehta, R.: On the existence of EFX under picky or non-differentiative agents. In: Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2024). pp. 2534–2536 (2024), extended abstract

A Proofs for Section 2

Full proofs of all reduction lemmas are in `proofs/lemmas.md` of [10]; we reproduce those used here.

Proof (of Lemma 1). Every other bundle lies in $M \setminus P$, so i values it at most $v_i(M \setminus P) \leq v_i(P) = v_i(X_i)$. Towards P , every other agent sees a single good or a bundle worthless to it. The remaining constraints are those of Y . $\square$

Lemma 4 (junk). *Let $J \neq \emptyset$ be the goods valued by no agent. If the instance without J has an EFX_0 allocation, so does the instance.*

Proof. Let Y be EFX_0 for $M \setminus J$. While the envy graph ($i \rightarrow j$ iff $v_i(Y_j) > v_i(Y_i)$) has a cycle, give each agent on it the bundle of its successor. The bundles are the same and no agent loses value; an agent that now sees its old bundle satisfies $v_i(Y_i \setminus \{h\}) \leq v_i(Y_i)$, so EFX_0 is preserved. The sum of values strictly increases, so this stops, with an acyclic envy graph. Let s be a source: $v_i(Y_s) \leq v_i(Y_i)$ for all i . Put $X_s = Y_s \cup J$. For $i \neq s$, $v_i(X_s \setminus \{h\}) \leq v_i(X_s) = v_i(Y_s) \leq v_i(Y_i)$; all other constraints are unchanged. $\square$

Proof (of Proposition 1). Induction on $n+m$ over 3-relevant instances with $1 \leq n \leq N_0$. One agent takes everything. Otherwise: (1) if some good is valued by nobody, use Lemma 4; (2) else if some agent has at most two relevant goods or is top-heavy, use rule R1 and Lemma 1; (3) else if some agent i has at least two private goods, let P be all of them: if $|P| = 3$, (i) is trivial, and if $P = \{x, y\}$ and $R_i \setminus P = \{z\}$, then $v_i(x) + v_i(y) \geq v_i(b_i) + v_i(c_i) > v_i(a_i) \geq v_i(z)$, so Lemma 1 applies (rule R2); (4) else the instance is a core, covered by hypothesis. Each case passes to a smaller 3-relevant instance. $\square$

In Lean this is `EFX.core_reduction` (with `EFX.peel`, `EFX.peelEmpty`, `EFX.peelR2` and `EFX.Inst.exists_efx0_of_junk`).

Proof (of Lemma 2). Fix i ; w_i is built from v_i alone. Goods outside R_i contribute 0 to both sides under both valuations, and removing goods common to S and T changes neither comparison, so it suffices that w_i preserves $v_i(X) \leq v_i(Y)$ for disjoint $X, Y \subseteq R_i$. Name the relevant goods with values $a \geq b \geq c > 0$. Comparisons with an empty side are decided by positivity; two single goods by the ties among a, b, c ; a single good against the other two by the sign of $a - (b + c)$, because $b < a + c$ and

$c < a + b$ always hold. Two disjoint pairs do not fit in three goods. Choose w_i on R_i (in the same order) by the pattern: (4, 3, 2) if $a > b > c$ and $a < b + c$; (3, 2, 1) if $a > b > c$ and $a = b + c$; (5, 2, 1) if $a > b > c$ and $a > b + c$; (2, 2, 1) if $a = b > c$; (3, 2, 2), (2, 1, 1), (3, 1, 1) if $a > b = c$ and $a < 2b$, $a = 2b$, $a > 2b$; (1, 1, 1) if $a = b = c$; (2, 1) or (1, 1) for two relevant goods; (1) for one; and $w_i(g) = 0$ for $g \notin R_i$. Each triple has the same ties and the same sign of $a - (b + c)$ as its pattern. $\square$

Lean: `EFX.112`, which uses all 31 permutations of these representatives (checked by `decide`) instead of sorting, and `EFX.efx0_iff_of_agree` for the transfer of EFX₀.

B Proofs for Section 3

These are the proofs of `proofs/lb_last_step.md` of [10], Sections 1–6.

Phase 1. (I1): i took its favourite remaining good, and goods leave G only by being picked. (I2): g was still in G at i 's turn. (I3): R1 steps process only agents with at most two goods left. (B): each block starts with an insertion step, when every unprocessed agent i has $R_i \subseteq G$; by (I1) g was picked before j 's turn, and not in an earlier block, since j was unprocessed when its own block started.

Proof (of Lemma 3). Since $|J| = S + \omega > S - \text{cap}(o)$, H extends to $C \subseteq J$ with $|C| = S - \text{cap}(o)$, which exactly fills the slots of the terminals other than o ; let $X_o = \text{base}(o) \cup (J \setminus C)$. If $x \notin U$, $x \neq o$, $Y_x = a_x$ had $\{b_x, c_x\} \subseteq X_o$, then $\{b_x, c_x\} \subseteq J \cup \text{base}(o)$, so $x \in E_o$, and $H \subseteq C$ meets $\{b_x, c_x\}$ outside X_o , a contradiction. $\square$

Lemma 5 (upgrades). *When the upgrade loop stops, (Y, U) is a valid pre-allocation, and (UT) no $k \notin U$ has $Y_k = b_k$, $c_k \in J$ and $b_k \notin NA$.*

Proof. (UT) is the stopping condition. (V1): by (I2) a junk good lies below the pick of every agent that values it, so it is in no N_i . (V2): c_u was junk; $b_u \notin NA$ held when u was upgraded, and NA only shrinks as U grows. $\square$

Proof (of Theorem 4). r exists: the first processed agent is inserted and holds its top, so it is not upgraded. (A1) $r \in T$: if $Y_r = \perp$, r is not frozen by definition; if $Y_r \in N_j$ for some $j \notin U$, then by (I1) j was processed after r , contradicting the choice of r . (A2) Every agent processed after r is in U . (A3) Let $x \in E_r$. Then x was processed before r (A2). At x 's turn, $a_x = Y_x$, every junk good, and Y_r were available, and $\{b_x, c_x\} \subseteq J \cup \{Y_r\}$,

so $R_x \subseteq G$ and x is a leader by (I3). Hence each block contains at most one exposed agent; let B^* be r 's block and k^* its exposed agent, if any. (A4) For a frozen x , some $j \notin U$ has $Y_x \in N_j$, and by (B) j is processed after x in x 's block. Iterating while the current agent is frozen gives a need chain of distinct agents of x 's block ending at a terminal. Let $\tau(x) = x$ if $x \in T$, and otherwise the end of some need chain from x ; it has $\text{cap} \geq 1$. For $x \in E_r$, $\pi_x = \{b_x, c_x\} \cap J \neq \emptyset$, since at most one of b_x, c_x is Y_r ; for the same reason no exposed pair lies in $\text{base}(r)$, the first condition of Lemma 3.

The agents $\tau(x)$, $x \in E_r \setminus B^*$, lie in distinct blocks other than B^* , so they are distinct terminals other than r , and $S - \text{cap}(r) \geq |E_r \setminus B^*|$. Let H contain one good of each π_x , a common one when two meet. If $E_r \cap B^* = \emptyset$, then $|H| \leq |E_r| \leq S - \text{cap}(r)$. Otherwise $E_r \cap B^* = \{k^*\}$, $k^* \neq r$ and $|E_r \setminus B^*| = |E_r| - 1$. If $k^* \in T$, or some need chain from k^* ends at a terminal other than r , then $\tau(k^*)$ can be chosen in $B^* \setminus \{r\}$, one more terminal, so $S - \text{cap}(r) \geq |E_r| \geq |H|$. If two sets π_x meet, then $|H| \leq |E_r| - 1 \leq S - \text{cap}(r)$. If none of these holds, the bad case holds. $\square$

Proof (of Theorem 5). Let $k = k^*$, $W = J \cup \{Y_r\}$; the new junk is $J' = W \setminus \{b_k, c_k\}$, and every pick other than Y_r is still a pick in P' . (a) P' is a pre-allocation: $Y'_{x_s} \in R_{x_s}$ since x_s needed it, and $b_k, c_k \in W$ are neither picks of P' nor goods c_u . (b) $NA' \subseteq NA$: $Y'_{x_s} \succ Y_{x_s}$ (or $Y_{x_s} = \perp$), so $N'_{x_s} \subseteq N_{x_s}$; k had $N_k = \emptyset$ and is now in U' . (c) Validity: $J' \subseteq W$, $J \cap NA = \emptyset$ and $Y_r \notin NA$ (A1), so $J' \cap NA' = \emptyset$; for $u \in U$, $b_u, c_u \notin NA \supseteq NA'$; $b_k, c_k \in W$ are not in NA . (d) A terminal $x \notin B^*$ keeps its pick and is a terminal of P' with the same slots. (e) $E'_k \subseteq E_r \setminus \{k\}$, where E'_k is taken in P' , with $J' \cup \{b_k, c_k\} \subseteq W$. An agent off the chain in E'_k is in E_r . An interior chain agent x_s ($1 \leq s < t$) is not: $a_{x_s} = Y'_{x_s} \succ Y_{x_s}$, so $Y_{x_s} \in \{b_{x_s}, c_{x_s}\}$, and $Y_{x_s} \notin W$. Nor is r : $Y_r \neq a_r$. If $Y_r = \perp$ or $Y_r = c_r$, then b_r was picked by another agent (I1), so $b_r \notin W$. If $Y_r = b_r$, then (UT) applies to r ($b_r \notin NA$ by (A1)), so $c_r \notin J$, and $c_r \neq Y_r$, so $c_r \notin W$. (f) If some $x \in E'_k$ had $\{b_x, c_x\} = \{b_k, c_k\}$, then $x \in E_r \setminus \{k\}$ and $\pi_x = \pi_k \neq \emptyset$, against disjointness. (g) By (f), every $x \in E'_k$ has a good of $\{b_x, c_x\}$ in J' ; let H' contain one each. By (e), $|H'| \leq |E_r| - 1 = |E_r \setminus B^*|$. The terminals $\tau(x)$, $x \in E_r \setminus B^*$, are distinct, outside B^* , and keep $\text{cap}' \geq 1$ by (d). So $|H'| \leq S' = S' - \text{cap}'(k)$, and Lemma 3 applies to P' and $o = k$. $\square$

Size of the large bundle (written, not formalized). In a valid pre-allocation each good of NA is the pick of exactly one frozen agent, so $|F| = |NA|$; counting goods gives $\omega = m - 2n + |NA|$, and the owner's bundle has $\omega + 2$ goods. The rotation does not increase NA .

C Algorithm K3ALG

Lists are in index order; “first” means smallest index. This is the program formalized as EFX.K3.algoC (proofs/k3_algorithm.md of [10]).

```

Stage R.  A := all agents;  M := all goods.
loop:
  if |A| <= 1: the agent of A takes M; stop.
  if M is empty: stop.
  i := the first agent of A for which, with p the
        first good of M with the largest vi(p),
        vi(p) = 0 or vi(M - p) <= vi(p);
  if there is none: go to Stage L.
  remove i from A; if vi(p) > 0, i takes p, and
    p is removed from M.
Stage L.  ai, bi, ci := i's goods sorted by value.
  order: repeatedly the first unprocessed agent with
        at most two of its goods left, else the
        first unprocessed agent.
  Phase 1 in this order; upgrades (first eligible k).
  if |J| <= S: complete without owner; stop.
  r := the last agent of the order not in U;
  E := the agents exposed for r;
  H := one junk good of each exposed pair, with one
        common good for the first meeting pair;
  if |H| <= S - cap(r): complete with owner r,
    placing H first; stop.
  k := the agent of E in r's block;
  chain: scan forward from k, appending each agent
        outside U that needs the current pick, while
        the current agent is frozen;
  rotate; recompute J, cap and S;
  complete without owner if |J| <= S,
    else with owner k.
    
```

Cost table (Lean bound with $N = n + m + 1$; finer bound written). Stage R: $15N^2$ per round, $O(n^2m)$ in all. Rankings: $14N^2$. Processing order: $15N^3$. Phase 1's picks: $17N^3$. Upgrades: $36N^4, O(n^4 + n^2m)$, the dominant term. Junk and slots: $O(N^3)$. Owner test: $20N^2 + 24N^3$. Blocks, k^* , chain, rotation, completion: $O(N^3)$. Output: $4N^2$. The total is at most $400N^4$ (EFX.K3.algoC_cost), also at most $6400(n + m)^4$. On random instances the count is far below the bound (ratio 0.002 at $n = 30$).

Evidence. Three implementations (a literal transcription of the Lean definitions, the fast Python version, and Lean’s own evaluation of `EFX.K3.algo`) agree on 200,000 random instances (the first two) and on 675 instances (all three), every output passes a raw EFX_0 check, and every Lean operation count is below the proved bound. The fast version takes 1.53s and 1.58s (medians) on random cores with 10^5 agents and 1.5×10^5 and 2×10^5 goods.

D The Lean Formalization

Trusted base. The file `lean/EFX/Model.lean`, copied from [9] with the namespace renamed, defines:

```
def finSum : (k : Nat) → (Fin k → Nat) → Nat
| 0, _ => 0
| k+1, f => finSum k (fun i => f i.castSucc)
           + f (Fin.last k)
structure Inst where
  n : Nat
  m : Nat
  v : Fin n → Fin m → Nat
abbrev Alloc := Fin I.m → Fin I.n
def bundleVal (X : I.Alloc) (i j : Fin I.n)
  (ex : Option (Fin I.m)) : Nat :=
  finSum I.m (fun g =>
    if X g = j ∧ ex ≠ some g then I.v i g else 0)
def EFX0 (X : I.Alloc) : Prop :=
  ∀ i j : Fin I.n, i ≠ j → ∀ g : Fin I.m, X g = j →
    I.bundleVal X i j (some g)
    ≤ I.bundleVal X i i none
def numRelevant (I : Inst) (i : Fin I.n) : Nat :=
  finSum I.m (fun g => if 0 < I.v i g then 1 else 0)
```

Main statements.

```
theorem EFX.target (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) :
  ∃ X : I.Alloc, I.EFX0 X
theorem EFX.LB.corollaryD (I : Inst) (hn : 0 < I.n)
  (h3 : ∀ i, numRelevant I i = 3)
  (hbal : ∀ i g, 2 * I.v i g ≤ finSum I.m (I.v i)) :
```

```

    ∃ X : I.Alloc, I.EFX0 X ∧ ∃ o, ∀ j, j ≠ o →
      finSum I.m (fun g => if X g = j then 1 else 0)
        ≤ 2
def EFX.K3.algo (I : Inst) (hn : 0 < I.n) : I.Alloc :=
  (algoC I hn).val
theorem EFX.K3.algo_efx0 (I : Inst) (hn : 0 < I.n)
  (h : ∀ i, numRelevant I i ≤ 3) :
  I.EFX0 (algo I hn)
theorem EFX.K3.algoC_cost (I : Inst) (hn : 0 < I.n) :
  (algoC I hn).cost ≤ 400 * (I.n + I.m + 1) ^ 4

```

`EFX.target_ordered` and `EFX.corollaryD_ordered` state the same over any type with `0`, `+` and `≤` satisfying the axioms of a linearly ordered cancellative commutative monoid (class `EFX.OrderedValue`), with nonnegativity as a hypothesis, relevance as $v_i(g) > 0$ and balance as $v_i(g) + v_i(g) \leq v_i(M)$; at $\mathbb{N}$ they are exactly the statements above.

Theorems cited (all in `lean/EFX/`; each has a `#print axioms` certificate listing at most the three standard axioms; ledger rows of [10] in brackets):

Result [ledger row]	Lean names (file in <code>lean/EFX/</code>)
Thm. 1 [T]	<code>EFX.target</code> (<code>Target</code>); <code>EFX.target_ordered</code> (<code>RealValues</code>)
Thm. 2 [D]	<code>EFX.LB.corollaryD</code> (<code>CorollaryD</code>); <code>EFX.corollaryD_ordered</code> (<code>RealValues</code>)
Thm. 7 [K3.ALG]	<code>EFX.K3.algo_efx0</code> (<code>K3Cost</code>)
Thm. [K3.ALG.TIME]	7 <code>EFX.K3.algoC_cost</code> (<code>K3CostBound</code>)
Lem. 1 [L2]	<code>EFX.peel</code> (<code>Peeling</code>); <code>EFX.peelEmpty</code> , <code>EFX.peelR2</code> (<code>PeelingR2</code>)
Lem. 4 [L3]	<code>EFX.Inst.exists_efx0_of_junk</code> (<code>Junk</code>)
Prop. 1 [T]	<code>EFX.core_reduction</code> (<code>Target</code>)
Lem. 2 [L12]	<code>EFX.l12</code> , <code>EFX.efx0_iff_of_agree</code> (<code>RealValues</code>)
Thm. 3 [S2.LB+]	<code>EFX.LB.Valid.sound</code> (<code>PreAlloc</code>)
Thm. 4 [S2.R]	<code>EFX.LB.theoremA_invalid</code> (<code>OwnerR</code>)
Thm. 5 [S2.LB+]	<code>EFX.LB.theoremB</code> (<code>Rotation</code>)
Thm. 6 [S2.LB+]	<code>EFX.LB.lbPlusRun_sound</code> , <code>EFX.LB.lbPlusOut_exists</code> (<code>LBPlus</code>)
Prop. 2 [K3.OWNER]	<code>EFX.LB.validOwner_iff</code> (<code>OwnerR</code>)
Audit [AUD]	<code>Audit.target_audit</code> , <code>Audit.corollaryD_audit</code> (<code>Audit</code>)
Second [K4.C4X.K3]	proof <code>EFX.C4min.corollaryD_K3</code> , <code>EFX.C4min.target_K3</code> (<code>K3Theorem</code>)

`EFX.LB.lbPlusRun_sound` quantifies over all runs of LB^+ (any Phase 1 order with R1 priority, any upgrade order, any need chain, any completion satisfying (OC)); `EFX.LB.lbPlusOut_exists` shows that an output always exists. The audit's second kernel (nanoda) checked 4,186 declarations: the 726 named declarations of the library at the time (the natural-number development of Theorems 1 and 2) and their dependencies. The files formalizing Theorems 1–7, their proofs and the audit have about 9,700 lines.

E Computational Certification

Each row is a ledger row of [10] with status CERTIFIED; certificates are stored allocations, re-checked without SAT from the raw EFX_0 definition, and the enumeration of connected cores is checked for completeness by a second, independent method.

Claim and scope	Size	Row
EFX ₀ exists, every 3-relevant in-stance, $n \leq 6$	251 + 3,182 connected cores	R1
EFX ₀ exists, every 3-relevant in-stance, $n \leq 7$	adds $n = 7$ (rows R2, R5 S2.N7)	
Theorem 2, every core with $n \leq 6$, connected or not	certificates, components combined	R3
Theorem 2, connected cores, $n = 7$, $11 \leq m \leq 13$	37, 541, 3,103 cores; 6 ⁷ files each	R2
Theorem 2, connected cores, $n = 8$, $m \in \{14, 15\}$	1,232 and 52 cores	R4
LB never fails, cores with $n \leq 6$	3,436 connected, 131 dis-connected	S2.N6
LB never fails, connected cores with $n = 7$	41,170 cores, 1.15×10^{10} pairs	S2.N7
LB never fails, connected cores, $n = 8$, $m \geq 13$	12,763 cores, 2.14×10^{10} pairs	S2.N8

For the rows on LB, the certificates (LB's outputs) confirm Theorem 2 in every labelling; that LB itself never fails holds for the labelling stored in the certificates, as LB breaks ties by index. Cores with $m = 2n$ are covered by a two-own-goods argument (every agent holding two of its own goods is safe).

Evidence (not proof). 5,844,263,904 runs of a literal implementation of LB⁺ (every core with $n \leq 6$ under every ranking profile and every sequence of insertion choices; random choices on larger cores; random non-core instances), asserting every lemma of Section 3 and checking every output against the raw definition: 1,327,101 rotations, 0 failures. An independent Python implementation gives identical tallies for $n \leq 5$.

Refutations (status REFUTED, with checkers):

- bundles of at most two goods suffice in a core: false for $n = 3$, $m = 5$ (Proposition 3(a), 14 of 216 profiles); no core with $n = 2$ fails;
- bundles of at most two goods suffice when $m \leq 2n - 2$: false, smallest $n = 4$, $m = 6$; at $n = 6$, $m = 10$, 57 of the 211 connected cores have a failing profile (confirmed by a second encoding);
- the same for instances that are not cores: four identical agents with values 4, 3, 2 on three goods, and three goods valued by nobody;
- one bundle of three goods suffices: false for $n = 4$, $m = 7$ (Proposition 3(b)); at $n = 6$, $m = 11$, 3 of 25 cores need four.

F Ordinality of Cores and the Counterexamples

A good is *alone* if it is the only good of its bundle.

Lemma 6 (cases of safety). *Let agent i have $R_i = \{a, b, c\}$ with $v_i(a) > v_i(b) > v_i(c) > 0$ and $v_i(a) < v_i(b) + v_i(c)$. Then i satisfies the EFX_0 constraints iff one of: (T) i holds a , and either i holds b or c , or b and c are not together in a bundle of at least three goods; (P) i holds b and c ; (B) i holds b , and a is alone; (C) i holds c , and a and b are alone; (E) a , b and c are all alone.*

Proof. The subset sums are ordered $0 < c < b < a < b + c < a + c < a + b < a + b + c$ (writing x for $v_i(x)$), so every constraint is decided by the ranking. Consider the goods of R_i that i holds. If it holds a and one more, every other bundle meets R_i in one good, worth $\leq b < a + c$. If it holds $\{b, c\}$, the only threat is $a < b + c$. If it holds only a : bundles meeting R_i in one good are harmless; $\{b, c\}$ as a bundle threatens $b < a$; b, c together with another good threaten $b + c > a$. If it holds only b : a not alone lies with c (threat a) or with a good outside R_i (threat $\geq a$), both $> b$; if a is alone, c 's bundle threatens at most $c < b$. If it holds only c , it needs a and b alone by the same argument. If it holds none of a, b, c , any bundle with a good of R_i and at least two goods threatens a positive amount, so all three must be alone. $\square$

The cases remain sufficient with ties, so tied cores reduce to strict ranking profiles (`proofs/lemmas.md`, L5).

Proposition 3 (limits of the shape). *Take strict rankings and strictly balanced values. (a) Let agents $0, 1, 2$ rank $g_0 \succ g_1 \succ p_i$, where p_i is private to agent i . No EFX_0 allocation has all bundles of at most two goods, while $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$ is EFX_0 . (b) Let agents $0, 1, 2, 3$ rank $g_0 \succ g_2 \succ p_0$, $g_0 \succ g_2 \succ p_1$, $g_1 \succ g_2 \succ p_2$, $g_1 \succ g_2 \succ p_3$. Every EFX_0 allocation with at most one bundle of more than two goods has bundle sizes $4, 1, 1, 1$.*

Exhaustive computation shows that no core with two agents needs a bundle of more than two goods, so (a) is a smallest example (ledger row X1 of [10]; `proofs/counterexamples.md` and `proofs/construction.md`, Section 1).

Proof (of Proposition 3). (a) With five goods in three bundles of at most two goods, the sizes are $2, 2, 1$, so exactly one good is alone. Good g_0 is everyone's top and has one holder, so two agents do not hold their top.

For them (T) fails, (C) and (E) need two or three alone goods, so each is in case (P) or (B), and both hold g_1 : impossible. In $\{g_0\}, \{g_1\}, \{p_0, p_1, p_2\}$, the holder of g_0 is in case (T), the holder of g_1 in case (B), and the third agent in case (C). (b) Each of the tops g_0, g_1 has a claimant that does not hold it (a *loser*). A loser is safe only by (P) or (B), which need it to hold g_2 , or by (C) or (E), which need g_2 alone. The two losers are distinct and cannot both hold g_2 , so g_2 is alone, and no agent is in case (P). A loser holding $\{g_2\}$ is in case (B) or (E) and needs its top alone; a loser not holding g_2 is in case (C) or (E) and needs its top and g_2 alone. So g_0, g_1, g_2 are alone, held by three agents, and the fourth bundle is $\{p_0, p_1, p_2, p_3\}$. The allocation giving g_0 to agent 0, g_1 to agent 2, g_2 to agent 1 and the private goods to agent 3 is EFX₀ (cases T, B, T, C). $\square$